PDF 2 DE 2 · PRÁTICA · PROJETO FINAL PET HASH & SORT · CENÁRIO B

Creche dos Bichos Hash & Sort, construída com TDD

O projeto pronto na pasta `untitled/`: o que cada arquivo faz, como foi construído teste a teste, como rodar, o que o programa mostra e como responder as 9 perguntas do relatório. Leia antes o PDF 1 (Teoria).

1 O que foi construído

```
untitled/
├── pom.xml            Java 21 + JUnit 5 (testes)
├── dados/pets.txt     50 bichinhos (20 + 30 novos)
└── src/
    ├── main/java/Pets/          o programa
    │   ├── Pet.java             ficha do bichinho
    │   ├── ArquivoPets.java     lê o pets.txt
    │   ├── TabelaHash.java      os canis (hash + lista)
    │   ├── ResultadoOrdenacao   comparações, trocas, tempo
    │   ├── Ordenacao.java       3 sorts + narração da fila
    │   ├── GeradorPets.java     1.000 e 10.000 bichinhos
    │   ├── GrandeCorrida.java   as 27 medições
    │   └── Main.java            menu da creche
    └── test/java/Pets/          48 testes JUnit
```

Cenário B · Creche dos Bichos. Durante o dia os bichinhos ficam em canis; o canil de cada um é calculado pelo **nome**. Quem cai no mesmo canil divide o espaço. No fim da tarde, a **fila da saída** segue o microchip.

Na creche	No código
Canis da creche	baldes da `TabelaHash` (`baldes[i]` = canil i)
Bichinhos dividindo o canil	lista de nós `No` (encadeamento à mão)
Lotação da creche	`fatorDeCarga()` e `situacaoDaLotacao()`
Fila da saída	`Pet[]` ordenado por microchip
Mudar o tamanho da creche	`reorganizar(M, funcao)`: rehashing
Troca de lugar na fila	`trocar(v, i, j)`, contado em `r.trocas`

RESPEITADO Nenhum `HashMap` / `HashSet` / `LinkedList` nos canis e nenhum `Arrays.sort` / `Collections.sort`, nem para montar o vetor "já ordenado" da corrida.

2 Como rodar

No IntelliJ (projeto já aberto):

1. Abra `untitled/pom.xml` e clique em **Load Maven Changes** (ícone do "m" que aparece no canto). Isso baixa o JUnit.
2. **Testes**: botão direito em `src/test/java` → **Run 'All Tests'**. Tem que aparecer **48 testes verdes**.
3. **Programa**: abra `Main.java` e clique no ▶ ao lado do `main`. A primeira linha deve ser "50 bichinhos chegaram na creche".

No terminal (PowerShell), dentro de `untitled`:

```
javac -d target/classes src/main/java/Pets/*.java
java -cp target/classes Pets.Main
```

O `javac` compila (rode de novo sempre que mudar o código) e o `java` abre o menu. Com Maven instalado, `mvn test` roda os 48 testes. O `Main` acha o `pets.txt` tanto rodando da pasta `untitled` quanto da pasta do projeto; se não achar, mostra o caminho completo onde procurou.

3 TDD: teste primeiro, código depois

TDD (*Test-Driven Development*, desenvolvimento guiado por testes) é escrever, antes do código, um pequeno programa que **confere** se o código faz o que deveria. O ciclo se repete para cada funcionalidade:

1. Vermelho
Escreve o teste do que ainda não existe e roda. Ele **falha** (às vezes nem compila): prova que o teste testa algo.

2. Verde
Escreve o código **mínimo** para o teste passar. Roda de novo todos os testes.

3. Refatorar
Melhora o código (nomes, organização) e roda os testes de novo: se continuarem verdes, nada quebrou.

Por que vale a pena aqui: os rastreios dos slides viram testes automáticos ({7, 2, 12, 8, 3} → 10 comparações e 5 trocas no Bubble), então se alguém do grupo mexer na contagem e errar, o teste avisa na hora. E quando o grupo trocou a história da clínica pela creche, os testes antigos garantiram que a tabela hash e os algoritmos continuaram funcionando.

JUnit em 30 segundos: cada método marcado com `@Test` é um teste. `assertEquals(esperado, obtido)` falha se forem diferentes; também usamos `assertTrue`, `assertFalse`, `assertNull` e `assertArrayEquals`. `@TempDir` cria uma pasta temporária (usada para testar a leitura de arquivo).

Ciclo	O teste exigia...	Testes
1 · `Pet`	`normalizar` e `toString`	4
2 · `TabelaHash`	hashes, inserir, buscar, remover, estatísticas	13
3 · `Ordenacao`	rastreios dos slides e fórmulas exatas	9
4 · gerador + corrida	chips únicos; `correr(...)` devolvendo resultados	5
5 · `ArquivoPets`	BOM, comentários, linha inválida, ≥ 50 bichinhos	4
6 · **Cenário B**	canis, lotação, narração da fila, menu e um dia inteiro na creche	7

Ciclo	O teste exigia...	Testes
7 · **auditoria**	opção 9 (trocar M e função) e n = 50 com os bichinhos do arquivo	6

Um ciclo de verdade (ciclo 1). Vocês já tinham o `Pet` com `getChaveNome()`. Primeiro veio o teste do que faltava:

PetTest.java

```
void normalizarServeParaONomeDigitadoNaBusca() {
    assertEquals("LUNA", Pet.normalizar("luna"));
    assertEquals("LUNA", Pet.normalizar("  Luna "));
    assertEquals("JOAO", Pet.normalizar("João"));
}
```

VERMELHO o Maven nem compilou:

```
[ERROR] PetTest.java:[28,33] cannot find symbol
  symbol:   method normalizar(java.lang.String)
  location: class Pets.Pet
[INFO] BUILD FAILURE
```

VERDE com o código mínimo no `Pet` (o `getChaveNome` passou a chamar este método):

Pet.java

```
public static String normalizar(String texto) {
    String semAcentos = Normalizer.normalize(texto.trim(),
Normalizer.Form.NFD)
            .replaceAll("\\p{M}", "");
    return semAcentos.toUpperCase(Locale.ROOT);
}
```

```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

`static` porque a busca precisa normalizar o nome **digitado** ("luna", " Luna ") sem ter um `Pet`. NFD separa "ç" em "c" + cedilha; `\p{M}` apaga as marcas de acento.

O ciclo 6: trocar a história para o Cenário B. A lógica não muda entre os cenários; muda o que aparece na tela. Então os testes do ciclo 6 **capturam a tela**: trocam o `System.out` por um "gravador", rodam o trecho e conferem o texto. O teste mais completo simula alguém digitando no teclado e roda o programa inteiro (por isso o `Scanner` passou a ser criado dentro do `main`).

CrecheTest.java

```
static String capturarTela(Runnable trecho) {
    PrintStream telaOriginal = System.out;
    ByteArrayOutputStream tela = new ByteArrayOutputStream();
    System.setOut(new PrintStream(tela, true,
StandardCharsets.UTF_8));
    try {
        trecho.run();
    } finally {
        System.setOut(telaOriginal);
    }
    return tela.toString(StandardCharsets.UTF_8);
}
```

CrecheTest.java

```
void umDiaNaCrecheDoComecoAoFim() {
    String tela = rodarPrograma(
            "4\n"              // mostrar os canis
            + "2\nluna\n"      // procurar as Lunas
            + "3\n13870255\n"  // a Luna cachorra foi para
casa
            + "5\n"            // lotação da creche
            + "abc\n"          // digitou errado
            + "0\n");          // fechar a creche
    assertTrue(tela.contains("50 bichinhos chegaram na
creche"), tela);
    assertTrue(tela.contains("Canil 5: "), tela);
    assertTrue(tela.contains("6 bichinho(s) encontrado(s)"),
tela);
    assertTrue(tela.contains("Tchau, Luna! Saiu do canil
5."), tela);
    assertTrue(tela.contains("superlotada"), tela);
    assertTrue(tela.contains("Digite apenas numeros."),
tela);
    assertTrue(tela.contains("Creche fechada"), tela);
}
```

VERMELHO primeiro nem compilou (`desenharCanis` e `situacaoDaLotacao` não existiam). Com os dois métodos criados "de mentira", compilou e os 7 testes da creche falharam **pelo motivo certo**: a tela ainda contava a história antiga. Os 35 testes antigos continuaram verdes.

```
[ERROR] Tests run: 7, Failures: 7 -- in Pets.CrecheTest
[ERROR]   CrecheTest.cadaBaldeEhUmCanil:56 Fichario: M = 13
baldes, 20 pets...
[ERROR]   CrecheTest.lotacaoDaCrecheSegueOFatorDeCarga:64
          expected: <tranquila> but was: <>
[ERROR]   CrecheTest.umDiaNaCrecheDoComecoAoFim:118 50 pacientes
carregados...
```

VERDE depois de escrever os canis, a lotação, a narração da fila e o menu da creche (e o **refatorar** de nomes e comentários, rodando tudo de novo):

```
[INFO] Tests run: 42, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

O segundo argumento do `assertTrue` é a tela inteira: quando o teste falha, ela aparece no relatório de erro. Depois, o ciclo 7 (seções 5 e 7) levou o total a 48 testes.

4 Dados e leitura do arquivo

`dados/pets.txt` tem uma linha por bichinho: `microchip;nome;especie;mania`. Linhas com `#` são comentários. Além dos 20 do apêndice, os 30 novos foram pensados para gerar colisões: **6 Lunas, 5 Thors, 5 Bobs, 3 Mels**, e o par **Amor e Roma** (anagramas, soma 303, colidem com a função soma em qualquer M). Troquem alguns pelos bichos de vocês, mantendo 8 dígitos e sem repetir microchip (o teste confere).

```
65502291;Paçoca;gato;caça o ponteiro laser
59302714;Amor;gata;só dorme no colo da visita
16748390;Roma;cachorra;late para pombos imaginários
```

ArquivoPets.java

```
linha = linha.replace("\uFEFF", "").trim();
if (linha.isEmpty() || linha.startsWith("#")) {
    continue;
}
String[] campos = linha.split(";", 4);
if (campos.length < 4) {
    System.out.println("Linha ignorada (faltam campos): " +
linha);
    continue;
}
try {
    int microchip = Integer.parseInt(campos[0].trim());
    temporario[quantidade] = new Pet(microchip,
campos[1].trim(),
                                     campos[2].trim(),
campos[3].trim());
    quantidade++;
} catch (NumberFormatException e) {
    System.out.println("Linha ignorada (microchip invalido):
" + linha);
}
```

`split(";", 4)` separa em no máximo 4 partes (a mania pode ter `;`); `parseInt` dentro de `try/catch` ignora microchip inválido sem derrubar o programa; o `replace("\uFEFF", "")` tira o caractere invisível que o Bloco de Notas põe no início.

5 Os canis: TabelaHash

Estrutura. Um vetor de canis; cada canil aponta para o primeiro nó de uma lista ligada feita à mão. O vetor começa todo `null` (canis vazios). `funcaoHash` escolhe entre a função 1 (soma) e a 2 (do grupo).

TabelaHash.java

```
private static class No {
    Pet pet;
    No proximo;

    No(Pet pet) {
        this.pet = pet;
        this.proximo = null;
    }
}

private No[] baldes;
private int quantidade;  // quantos bichinhos estão na creche
private int funcaoHash;

public TabelaHash(int m, int funcaoHash) {
    this.baldes = new No[m];
    this.quantidade = 0;
    this.funcaoHash = funcaoHash;
}
```

As duas funções hash. Recebem a chave já normalizada e devolvem 0 a M − 1. Como os nomes são curtos e em maiúsculas, a soma nunca estoura o `int`, então `%` basta.

TabelaHash.java

```
public static int hashSoma(String chave, int m) {
    int soma = 0;
    for (int i = 0; i < chave.length(); i++) {
        soma += chave.charAt(i);
    }
    return soma % m;
}

// FUNÇÃO HASH 2 (do grupo): soma PONDERADA - cada letra pesa
conforme a posição
public static int hashPonderada(String chave, int m) {
    int soma = 0;
    for (int i = 0; i < chave.length(); i++) {
        soma += chave.charAt(i) * (i + 1);
    }
    return soma % m;
}
```

Inserir (matricular): recusa microchip repetido (o microchip não é a chave, então a checagem olha todos os canis), calcula o canil pelo nome e coloca o bichinho no **início** da lista (dois passos, nessa ordem).

TabelaHash.java

```
public boolean inserir(Pet pet) {
    if (buscarPorMicrochip(pet.getMicrochip()) != null) {
        return false;
    }
    int i = indice(pet.getChaveNome());
    No novo = new No(pet);
    novo.proximo = baldes[i];
    baldes[i] = novo;
    quantidade++;
    return true;
}
```

Buscar por nome devolve **todos** os bichinhos com aquele nome (não para na primeira Luna). Vai direto ao canil e percorre só ele, duas vezes: uma para contar, outra para preencher um vetor do tamanho certo.

TabelaHash.java

```
public Pet[] buscarPorNome(String nome) {
    String chave = Pet.normalizar(nome);
    No inicio = baldes[indice(chave)];

    // 1a volta no canil: contar quantos bichinhos têm essa
chave
    int encontrados = 0;
    No atual = inicio;
    while (atual != null) {
        if (atual.pet.getChaveNome().equals(chave)) {
            encontrados++;
        }
```

Remover por microchip (foi para casa): `anterior` e `atual` andam juntos; achou, "costura" a lista por cima do nó.

TabelaHash.java

```
public Pet removerPorMicrochip(int microchip) {
    for (int i = 0; i < baldes.length; i++) {
        No anterior = null;
        No atual = baldes[i];
        while (atual != null) {
            if (atual.pet.getMicrochip() == microchip) {
                if (anterior == null) {
                    baldes[i] = atual.proximo;
                } else {
                    anterior.proximo = atual.proximo;
                }
                quantidade--;
                return atual.pet;
            }
            anterior = atual;
            atual = atual.proximo;
        }
    }
    return null;
}
```

Desenhar os canis monta o texto num `StringBuilder` (um "rascunho" de texto) em vez de imprimir direto: assim o teste confere o desenho e o `exibir()` só faz `System.out.print(desenharCanis())`.

TabelaHash.java

```
public String desenharCanis() {
    StringBuilder sb = new StringBuilder();
    sb.append("Creche: " + baldes.length + " canis, " +
quantidade
            + " bichinhos, funcao hash " + funcaoHash +
"\n");
    for (int i = 0; i < baldes.length; i++) {
        sb.append("Canil " + i + ": ");
        No atual = baldes[i];
        if (atual == null) {
            sb.append("(vazio)\n");
            continue;
        }
        while (atual != null) {
            sb.append(atual.pet.getNome() + " (" +
atual.pet.getMicrochip() + ") -> ");
            atual = atual.proximo;
        }
        sb.append("null\n");
    }
    return sb.toString();
}
```

Mudar os canis (ciclo 7, opção 9). A auditoria mostrou que M = 17 e M = 31 só apareciam nas estatísticas. Agora o menu deixa trocar o M e a função da creche: `reorganizar` guarda todos os bichinhos e reinsere cada um, porque o canil depende de M e da função (é o **rehashing** da teoria). O teste exige que o resultado seja igual a uma creche que já nascesse com M = 17 e a função 2.

TabelaHash.java

```
public void reorganizar(int novoM, int novaFuncao) {
    Pet[] todos = todosOsPets();
    baldes = new No[novoM];
    funcaoHash = novaFuncao;
    quantidade = 0;
    for (Pet p : todos) {
        inserir(p);
    }
}
```

```
        atual = atual.proximo;
    }

    // 2a volta: copiar esses bichinhos para um vetor do
tamanho certo
    Pet[] resultado = new Pet[encontrados];
    int k = 0;
    atual = inicio;
    while (atual != null) {
        if (atual.pet.getChaveNome().equals(chave)) {
            resultado[k] = atual.pet;
            k++;
        }
        atual = atual.proximo;
    }
    return resultado;
}
```

Lotação da creche = fator de carga, `(double) quantidade / M` (o cast evita divisão inteira). A situação usa o limite dos slides (0,75); acima de 1 há mais bichinho que canil.

TabelaHash.java

```
public String situacaoDaLotacao() {
    double lotacao = fatorDeCarga();
    if (lotacao <= 0.75) {
        return "tranquila";
    } else if (lotacao <= 1.0) {
        return "no limite";
    }
    return "superlotada";
}
```

Colisões: a contagem simples soma (tamanho − 1) por canil; `colisoesEntreNomesDiferentes()` soma (nomes distintos − 1), a definição técnica. O teste que fixou as estatísticas usa os 20 do apêndice, contados à mão antes:

TabelaHashTest.java

```
void estatisticasDaTurminhaComM13() {
    TabelaHash t = crecheCom(turminha(), 13, 1);
    assertEquals(20.0 / 13, t.fatorDeCarga(), 1e-9);
    assertEquals(3, t.baldesVazios());
    assertEquals(5, t.maiorLista());
    assertEquals(10, t.colisoes());
    assertEquals(6, t.colisoesEntreNomesDiferentes()); // só
nomes diferentes contam
}
```

TabelaHashTest.java

```
void reorganizarRecalculaOCanilDeTodos() {
    TabelaHash t = crecheCom(turminha(), 13, 1);
    t.reorganizar(17, 2);
    TabelaHash nova = crecheCom(turminha(), 17, 2);
    assertEquals(17, t.getM());
    assertEquals(20, t.getQuantidade());
    assertEquals(3, t.buscarPorNome("Luna").length);
    assertEquals(nova.baldesVazios(), t.baldesVazios());
    assertEquals(nova.maiorLista(), t.maiorLista());
    assertEquals(nova.colisoes(), t.colisoes());
    assertEquals(TabelaHash.hashPonderada("LUNA", 17),
t.canilDoNome("Luna"));
    assertTrue(t.desenharCanis().startsWith("Creche: 17
canis, 20 bichinhos, funcao hash 2"));
}
```

```
Quantos canis (M)? Funcao hash (1 = soma dos codigos, 2 = ponderada do grupo): Creche reorganizada: 17 canis, funcao hash 2. Lotacao:
2,94 (superlotada)


Creche: 17 canis, 50 bichinhos, funcao hash 2
Canil 0: Roma (16748390) -> null
Canil 1: Pandora (76690431) -> Mel (22091187) -> Mel (54017263) -> Mel (23674018) -> null
Canil 2: Amor (59302714) -> null
Canil 3: Maya (34587019) -> null
Canil 4: Nina (18345602) -> Nina (95102846) -> Amora (51200468) -> null
Canil 5: Chico (33208179) -> Chico (83950472) -> null
Canil 6: (vazio)
Canil 7: Toddy (62015547) -> Paçoca (65502291) -> Paçoca (27365081) -> null
Canil 8: Lola (27719384) -> Lola (25089734) -> Tobias (78603425) -> Bidu (80265143) -> null
Canil 9: Luna (60218841) -> Luna (13870255) -> Luna (29573006) -> Luna (47120395) -> Luna (85301726) -> Luna (19274635) -> Pretinha
(43910578) -> null
Canil 10: (vazio)
Canil 11: Thor (31057720) -> Thor (57741820) -> Thor (12093847) -> Thor (66489012) -> Thor (56481920) -> Pipoca (48812903) -> Pipoca
(64829310) -> null
Canil 12: Max (71520936) -> Belinha (40623187) -> Simba (90027758) -> Simba (87412650) -> null
Canil 13: Frida (44183067) -> Frida (32719864) -> null
Canil 14: Bob (39021746) -> Bob (84460013) -> Bob (30958174) -> Bob (92846105) -> Bob (74036518) -> null
Canil 15: Kiara (50738291) -> null
Canil 16: Nala (38841052) -> Fred (81196354) -> Fred (11594068) -> Zeus (70334512) -> Zeus (69157803) -> null
```

Com a função 2 e M = 17, as 6 Lunas ficam no canil 9 e os 5 Thors no 11: nome repetido continua junto (mesma chave), mas os nomes diferentes se espalham mais.

O programa rodando: opção 4 (os canis) e opção 5 (lotação) com os 50 bichinhos

```
Creche: 13 canis, 50 bichinhos, funcao hash 1
Canil 0: Simba (87412650) -> Simba (90027758) -> null
Canil 1: Mel (23674018) -> Mel (54017263) -> Mel (22091187) -> null
Canil 2: Zeus (69157803) -> Pipoca (64829310) -> Zeus (70334512) -> Pipoca (48812903) -> null
Canil 3: Fred (11594068) -> Bob (74036518) -> Bob (92846105) -> Bob (30958174) -> Fred (81196354) -> Bob (84460013) -> Bob (39021746) -
> null
Canil 4: Roma (16748390) -> Amor (59302714) -> Amora (51200468) -> null
Canil 5: Belinha (40623187) -> Pretinha (43910578) -> Thor (56481920) -> Thor (66489012) -> Thor (12093847) -> Luna (19274635) -> Luna
(85301726) -> Luna (47120395) -> Luna (29573006) -> Thor (57741820) -> Luna (13870255) -> Thor (31057720) -> Luna (60218841) -> null
Canil 6: Bidu (80265143) -> null
Canil 7: Frida (32719864) -> Chico (83950472) -> Paçoca (27365081) -> Chico (33208179) -> Frida (44183067) -> Paçoca (65502291) -> null
Canil 8: Tobias (78603425) -> Nina (95102846) -> Nina (18345602) -> null
Canil 9: Kiara (50738291) -> Max (71520936) -> null
Canil 10: Lola (25089734) -> Maya (34587019) -> Pandora (76690431) -> Lola (27719384) -> null
Canil 11: Nala (38841052) -> Toddy (62015547) -> null
Canil 12: (vazio)
```

```
--- Lotacao da creche ---
Lotacao da creche ..... 3,85 bichinhos por canil (50 / 13):
superlotada
Canis vazios .......... 1
Canil mais cheio ...... 13 bichinhos (maior lista)
Colisoes (bichinho em canil ocupado) ... 38
Colisoes entre nomes diferentes ........ 14
```

O canil 5 junta 6 Lunas, 5 Thors, Belinha e Pretinha; o canil 4 junta Amor, Roma e Amora. Com 50 bichinhos em 13 canis, a creche está superlotada.

```
--- Comparando tamanhos (M) e funcoes hash com os mesmos 50
bichinhos ---
Funcao     M  Lotacao  Vazios  Maior  Colisoes  Nomes dif.
1         13     3,85       1     13        38          14
1         17     2,94       3     11        36          12
1         31     1,61      15     11        34          10
2         13     3,85       0      9        37          13
2         17     2,94       2      7        35          11
2         31     1,61      15      9        34          10
```

A opção 5 também monta creches de teste com os **mesmos** 50 bichinhos para M = 13, 17, 31 e as duas funções (comparação justa).

6 A fila da saída: Ordenacao

Os testes vieram dos slides. Se a contagem estiver no lugar errado (por exemplo, contando o teste `j >= 0` como comparação), o número muda e o teste fica vermelho. Também testados: fórmulas exatas para n = 50 já ordenado e inverso (ex.: Quick ordenado = 1.225 comparações e 0 trocas), trocas do Bubble = deslocamentos do Insertion, a cópia não altera o original, os 3 organizam a turminha e a narração usa as palavras da creche ("trocam de lugar na fila", "anda um lugar para trás", "lado de quem sai depois").

OrdenacaoTest.java

```java
void bubbleNoVetorDosSlides() {
    Pet[] v = fila(7, 2, 12, 8, 3);
    ResultadoOrdenacao r = Ordenacao.bubbleSort(v, false);
    assertArrayEquals(new int[] {2, 3, 7, 8, 12},
microchips(v));
    assertEquals(10, r.comparacoes);
    assertEquals(5, r.trocas);
}
```

OrdenacaoTest.java

```java
void quickNoVetorDosSlides() {
    Pet[] v = fila(7, 1, 3, 5, 2, 6, 4);
    ResultadoOrdenacao r = Ordenacao.quickSort(v, false);
    assertArrayEquals(new int[] {1, 2, 3, 4, 5, 6, 7},
microchips(v));
    assertEquals(11, r.comparacoes);   // 6 + 2 + 2 + 1
    assertEquals(6, r.trocas);
}
```

Bubble, com a narração da fila da saída (`narrar = true` só no passo a passo) e a parada antecipada `houveTroca`:

Ordenacao.java

```
for (int i = 0; i < n - 1; i++) {
    boolean houveTroca = false;
    long compAntes = r.comparacoes, trocasAntes = r.trocas;
    if (narrar) System.out.println("\n== Bubble Sort - passagem " + (i + 1) + " ==");

    for (int j = 0; j < n - 1 - i; j++) {
        r.comparacoes++;
        if (v[j].getMicrochip() > v[j + 1].getMicrochip()) {
            if (narrar) System.out.println("  " + ficha(v[j]) + " > " + ficha(v[j + 1])
                    + "? SIM -> trocam de lugar na fila!");
            trocar(v, j, j + 1);
            r.trocas++;
            houveTroca = true;
        } else if (narrar) {
            System.out.println("  " + ficha(v[j]) + " > " + ficha(v[j + 1])
                    + "? NAO -> " + v[j].getNome() + " fica onde esta.");
        }
    }

    if (narrar) {
        System.out.println("  fim da passagem " + (i + 1) + " -> comparacoes: "
                + (r.comparacoes - compAntes) + " | trocas: " + (r.trocas - trocasAntes));
        mostrarFila(v);
    }
    if (!houveTroca) {
        if (narrar) System.out.println("  Ninguem trocou de lugar: a fila da saida esta organizada!");
        break;
    }
}
return r;
```

Insertion: o `while (j >= 0 && v[j] > chave)` do Material de Apoio foi separado em dois para contar **só** a comparação entre microchips; cada `v[j + 1] = v[j]` é um **deslocamento** (o bichinho anda um lugar para trás na fila; guardado em `r.trocas`).

Ordenacao.java

```
while (j >= 0) {
    r.comparacoes++;
    if (v[j].getMicrochip() > chave.getMicrochip()) {
        if (narrar) System.out.println("  " + ficha(v[j]) + " > " + ficha(chave)
                + "? SIM -> " + v[j].getNome() + " anda um lugar para tras (posicao "
                + (j + 1) + ")");
        v[j + 1] = v[j];                    // DESLOCAMENTO
        r.trocas++;
        j--;
    } else {
        if (narrar) System.out.println("  " + ficha(v[j]) + " > " + ficha(chave)
                + "? NAO -> achou o lugar.");
        break;
    }
}
v[j + 1] = chave;
```

Quick: partição com pivô = último (como nos slides). Os menores que o pivô vão para o "lado de quem sai antes". Só conta (e só faz) a troca quando `i != j`; senão, uma fila já ordenada mostraria milhões de "trocas" sem ninguém sair do lugar. O `quickSortRecursivo` chama a partição e depois a si mesmo para os dois lados.

Ordenacao.java

```
Pet pivo = v[fim];
int i = inicio - 1;
if (narrar) System.out.println("\n== Quick Sort - posicoes " + inicio + " a " + fim
        + " da fila | pivo: " + ficha(pivo) + " ==");

for (int j = inicio; j < fim; j++) {
    r.comparacoes++;
    if (v[j].getMicrochip() < pivo.getMicrochip()) {
        i++;
        if (i != j) {
            if (narrar) System.out.println("  " + ficha(v[j])
                    + " < pivo? SIM -> troca de lugar com " + ficha(v[i]));
            trocar(v, i, j);
            r.trocas++;
        } else if (narrar) {
            System.out.println("  " + ficha(v[j])
                    + " < pivo? SIM -> ja esta do lado de quem sai antes");
        }
    } else if (narrar) {
        System.out.println("  " + ficha(v[j]) + " < pivo? NAO -> fica do lado de quem sai depois");
```

```
    }
}
if (i + 1 != fim) {
    trocar(v, i + 1, fim);
    r.trocas++;
}
if (narrar) {
    System.out.println("  " + pivo.getNome() + " vai para a posicao " + (i + 1)
            + " (lugar definitivo) | total -> comparacoes: " + r.comparacoes
            + " | trocas: " + r.trocas);
    mostrarFila(v);
}
return i + 1;
```

Modo passo a passo (opção 7, até 10 bichinhos), com os 6 primeiros da creche:

```
== Bubble Sort - passagem 1 ==
  Simba (87412650) > Simba (90027758)? NAO -> Simba fica onde esta.
  Simba (90027758) > Mel (23674018)? SIM -> trocam de lugar na fila!
  Simba (90027758) > Mel (54017263)? SIM -> trocam de lugar na fila!
  Simba (90027758) > Mel (22091187)? SIM -> trocam de lugar na fila!
  Simba (90027758) > Zeus (69157803)? SIM -> trocam de lugar na fila!
  fim da passagem 1 -> comparacoes: 5 | trocas: 4
  Fila: 0:Simba(87412650) 1:Mel(23674018) 2:Mel(54017263) 3:Mel(22091187) 4:Zeus(69157803) 5:Simba(90027758)

== Bubble Sort - passagem 2 ==
  Simba (87412650) > Mel (23674018)? SIM -> trocam de lugar na fila!
  Simba (87412650) > Mel (54017263)? SIM -> trocam de lugar na fila!
  Simba (87412650) > Mel (22091187)? SIM -> trocam de lugar na fila!
  Simba (87412650) > Zeus (69157803)? SIM -> trocam de lugar na fila!
  fim da passagem 2 -> comparacoes: 4 | trocas: 4
```

7 Gerador de bichinhos e a Grande Corrida

Microchips únicos sem ordenação pronta: começa em 10.000.000 e soma um passo sorteado de 1 a 8.999. Sai crescente e sem repetir; o maior possível com 10.000 bichinhos é 99.990.000 (8 dígitos). `new Random(2026)` (semente fixa) gera sempre os mesmos dados, bom para repetir medições.

GeradorPets.java

```
public static Pet[] gerarOrdenados(int n) {
    Pet[] pets = new Pet[n];
    int microchip = 10_000_000;
    for (int i = 0; i < n; i++) {
        microchip += 1 + sorteio.nextInt(8999);
        String nome = NOMES[sorteio.nextInt(NOMES.length)];
        String especie =
ESPECIES[sorteio.nextInt(ESPECIES.length)];
        String mania =
MANIAS[sorteio.nextInt(MANIAS.length)];
        pets[i] = new Pet(microchip, nome, especie, mania);
    }
    return pets;
}
```

A partir dele: **ordem inversa** = ler de trás para frente; **aleatório** = embaralhar uma cópia (Fisher-Yates: do fim para o começo, troca cada posição i com uma sorteada entre 0 e i).

GeradorPets.java

```
public static Pet[] embaralhado(Pet[] base) {
    Pet[] v = Ordenacao.copiar(base);
    for (int i = v.length - 1; i > 0; i--) {
        int j = sorteio.nextInt(i + 1);
        Pet aux = v[i];
        v[i] = v[j];
        v[j] = aux;
    }
    return v;
}
```

O pior caso do Quick estoura a pilha. Com 10.000 bichinhos já ordenados, são 10.000 chamadas recursivas empilhadas. Nos testes, com pilha de 512 KB deu `StackOverflowError`. A solução, sem mudar o algoritmo dos slides: rodar a corrida numa *thread* com pilha de 256 MB.

Main.java

```
static void grandeCorrida() {
    Thread corrida = new Thread(null, () ->
GrandeCorrida.executar(doArquivo),
            "corrida", 256L * 1024 * 1024);
    corrida.start();
    try {
        corrida.join();
    } catch (InterruptedException e) {
        System.out.println("Corrida interrompida.");
    }
}
```

Ciclo 4 do TDD: o teste exigia um método `correr(tamanhos, repeticoes)` que **devolve** os resultados (o `executar()` do menu só imprimia, impossível de testar). O teste ficou vermelho por "cannot find symbol: correr" e a corrida foi refatorada.

n = 50 com os bichinhos do grupo (ciclo 7). O enunciado diz que o gerador serve para 1.000 e 10.000 e que a comparação é "com bichinhos criados pelo grupo". Por isso `montarEntradas` usa os 50 do `pets.txt` no n = 50: copia, coloca em ordem com o **nosso** Insertion Sort (fora do cronômetro, sem `Arrays.sort`) e daí tira as três situações.

GrandeCorrida.java

```java
public static Pet[][] montarEntradas(Pet[] doGrupo, int tamanho) {
    Pet[] crescente;
    if (usaOGrupo(doGrupo, tamanho)) {
        crescente = new Pet[tamanho];
        for (int i = 0; i < tamanho; i++) {
            crescente[i] = doGrupo[i];
        }
        Ordenacao.insertionSort(crescente, false);
    } else {
        crescente = GeradorPets.gerarOrdenados(tamanho);
    }
    return new Pet[][] {
        GeradorPets.embaralhado(crescente),
        crescente,
        GeradorPets.invertido(crescente)
    };
}
```

Medição justa: cópia fora do cronômetro, sem narração, 5 repetições com média e uma rodada de aquecimento antes (compilador JIT); no fim, confere se ordenou.

GrandeCorrida.java

```java
private static ResultadoOrdenacao medir(int algoritmo, Pet[] entrada, int repeticoes) {
    ResultadoOrdenacao r = null;
    long somaTempos = 0;
    for (int rep = 0; rep < repeticoes; rep++) {
        Pet[] copia = Ordenacao.copiar(entrada);

        long inicio = System.nanoTime();
        r = Ordenacao.ordenar(algoritmo, copia, false); // narrar = false!
        long fim = System.nanoTime();

        somaTempos += fim - inicio;
        if (!Ordenacao.estaOrdenado(copia)) {
            System.out.println("ERRO: " + Ordenacao.NOMES[algoritmo] + " nao ordenou!");
        }
    }
    r.tempoNanos = somaTempos / repeticoes;
    return r;
}
```

GrandeCorridaTest.java

```java
ResultadoOrdenacao[][][] res = GrandeCorrida.correr(new int[] {n}, 1);
int todos = n * (n - 1) / 2;
// [algoritmo][situacao][tamanho]  situacao: 0 aleatório, 1 já ordenado, 2 inversa
assertEquals(n - 1, res[0][1][0].comparacoes);
assertEquals(todos, res[0][2][0].trocas);
assertEquals(0, res[1][1][0].trocas);
assertEquals(todos, res[1][2][0].comparacoes);
assertEquals(todos, res[2][1][0].comparacoes);
assertEquals(todos, res[2][2][0].comparacoes);
assertEquals(res[0][0][0].trocas, res[1][0][0].trocas);
```

Resultado da opção 8 (n = 50: os bichinhos do arquivo; tempos desta máquina, os de vocês vão variar, as contagens não)

```
TEMPO MEDIO (ms)
Algoritmo       Situacao             n = 50     n = 1.000    n = 10.000
Bubble Sort     Aleatorio             0,002         2,424        60,919
Bubble Sort     Ja ordenado           0,001         0,005         0,006
Bubble Sort     Ordem inversa         0,070         0,605        56,557
Insertion Sort  Aleatorio             0,026         0,183        18,084
Insertion Sort  Ja ordenado           0,003         0,002         0,016
Insertion Sort  Ordem inversa         0,063         0,389        38,778
Quick Sort      Aleatorio             0,001         0,035         0,552
Quick Sort      Ja ordenado           0,002         0,314        37,017
Quick Sort      Ordem inversa         0,002         0,264        28,747

COMPARACOES
Algoritmo       Situacao             n = 50     n = 1.000    n = 10.000
Bubble Sort     Aleatorio              1219        498225      49994097
Bubble Sort     Ja ordenado              49           999          9999
Bubble Sort     Ordem inversa          1225        499500      49995000
Insertion Sort  Aleatorio               703        252031      24890932
Insertion Sort  Ja ordenado              49           999          9999
Insertion Sort  Ordem inversa          1225        499500      49995000
Quick Sort      Aleatorio               238         10111        158262
Quick Sort      Ja ordenado            1225        499500      49995000
Quick Sort      Ordem inversa          1225        499500      49995000
```

```
TROCAS (no Insertion Sort: deslocamentos)
Algoritmo       Situacao              n = 50     n = 1.000    n = 10.000
Bubble Sort     Aleatorio               658        251037      24880941
Bubble Sort     Ja ordenado               0             0             0
Bubble Sort     Ordem inversa          1225        499500      49995000
Insertion Sort  Aleatorio               658        251037      24880941
Insertion Sort  Ja ordenado               0             0             0
Insertion Sort  Ordem inversa          1225        499500      49995000
Quick Sort      Aleatorio                92          4507         61483
Quick Sort      Ja ordenado               0             0             0
Quick Sort      Ordem inversa            25           500          5000
```

8 O menu e a demonstração ao vivo

```
+========================================+
|            CRECHE DOS BICHOS           |
+========================================+
| 1 - Matricular bichinho                |
| 2 - Procurar bichinhos por nome        |
| 3 - Bichinho foi para casa (microchip) |
| 4 - Mostrar os canis (tabela hash)     |
| 5 - Lotacao da creche (estatisticas)   |
| 6 - Organizar a fila da saida          |
| 7 - Fila da saida passo a passo        |
| 8 - A Grande Corrida                   |
| 9 - Mudar os canis (M e funcao hash)   |
| 0 - Fechar a creche                    |
+========================================+
```

```
Nome do bichinho: Chave LUNA -> canil 5
6 bichinho(s) encontrado(s):
  Luna (coelha) - microchip 19274635 - mania: rói o rodapé da
cozinha
  Luna (gata) - microchip 85301726 - mania: dorme dentro da
máquina de lavar
  Luna (cachorra) - microchip 47120395 - mania: late para a
própria sombra
  Luna (calopsita) - microchip 29573006 - mania: assobia quando
alguém chega
  Luna (cachorra) - microchip 13870255 - mania: rouba meias
  Luna (gata) - microchip 60218841 - mania: dorme em cima do
teclado


Nome do bichinho: Chave REX -> canil 5
Nenhum bichinho com esse nome na creche.


Microchip do bichinho que foi para casa: Tchau, Luna! Saiu do
canil 5.


Microchip (8 digitos): Nome: Especie: Mania: Bichinho
matriculado! Chave BOLINHA -> canil 2
```

O `Main` lê cada entrada com `nextLine()` e converte com `Integer.parseInt` (evita a armadilha do `nextInt`); letra no lugar de número mostra "Digite apenas numeros" e pergunta de novo.

Roteiro de 6 minutos para a apresentação

1. Rodar os **48 testes** no IntelliJ (todos verdes) e explicar o ciclo vermelho → verde com o exemplo do `Pet` ou da creche.
2. **4**: mostrar o canil 5 lotado de Lunas e Thors.
3. **2** "luna": as 6 Lunas. Depois "Rex": cai no canil 5 e não acha ninguém.
4. **1**: matricular o bichinho de alguém do grupo; **3**: um bichinho vai para casa.
5. **5**: lotação "superlotada" e a comparação de M e funções; depois **9** com M = 17 e função 2, e **4** de novo para ver os canis mais equilibrados.
6. **7**: fila da saída passo a passo com 5 bichinhos (ou o bônus com pelúcias narrando o Bubble).
7. **8**: a corrida, destacando o Quick no aleatório e o Quick na fila já ordenada.

Todos do grupo precisam saber explicar qualquer parte: usem o PDF 1 e os testes como roteiro de estudo.

9 As 9 perguntas do relatório, com os números do programa

Os números abaixo saíram das opções 5 e 8 com os nossos 50 bichinhos. Se vocês trocarem bichinhos no `pets.txt`, rodem de novo e atualizem; os tempos devem vir do computador de vocês.

1. Por que todas as Lunas caem no mesmo balde (canil), qualquer que seja M? É colisão?
Porque têm a **mesma chave** (LUNA) e a função é sempre a mesma conta: soma 304, então 304 % 13 = 5, 304 % 17 = 15, 304 % 31 = 25, sempre o mesmo canil para todas. **Não é colisão** no sentido técnico (colisão é chave *diferente* no mesmo índice), mas as 6 Lunas dividem o mesmo canil. Por isso o programa mostra duas contagens: com M = 13, 38 bichinhos entraram em canil ocupado, mas só 14 casos são de nomes diferentes.

2. Exemplo de colisão de verdade. Com M = 13 e a função soma: **LUNA** (304) e **THOR** (317) no canil 5; **Amor**, **Roma** e **Amora** no canil 4 (Amor e Roma são anagramas, soma 303);

5. 10.000 bichinhos numa creche com M = 17? A lotação (fator de carga) vai a **588,24** bichinhos por canil: superlotada. Medido com os 10.000 do gerador: canil mais cheio com 2.484 (função 1); procurar um nome inexistente ("Rex") percorre o canil inteiro, **1.915** comparações, quase uma busca sequencial. **Solução**: aumentar M (um primo perto de n ÷ 0,75, por exemplo 10.007 ou 13.337) e **reinserir** todos (rehashing). Com M = 10.007, a lotação cai para 1,00 e o "Rex" cai num canil vazio (9.992 vazios): busca O(1). **Mas** o gerador usa só 16 nomes: as 630 Lunas continuam no mesmo canil, porque é a mesma chave, e a busca por "Luna" precisa listar as 630 de qualquer jeito. E com a função soma, Frida e Chico seguem juntos (maior lista 1.263); com a ponderada, a maior lista cai para 652.

Frida, **Chico** e **Paçoca** no canil 7 (Frida e Chico somam 358 e colidem em *qualquer* M).

3. Qual tamanho gerou menos colisões? Por que primos?

Função	M	Lotação	Vazios	Maior	Colisões	Nomes dif.
1 soma	13	3,85	1	13	38	14
1 soma	17	2,94	3	11	36	12
1 soma	31	1,61	15	11	34	10
2 ponderada	13	3,85	0	9	37	13
2 ponderada	17	2,94	2	7	35	11
2 ponderada	31	1,61	15	9	34	10

M = 31 (o tamanho escolhido pelo grupo) teve menos colisões (34, só 10 entre nomes diferentes), à custa de 15 canis vazios. Entre 13 e 17, o 17 foi melhor. Primos ajudam porque não têm divisores em comum com padrões das somas (com M = 10, só o último dígito da soma importaria). Mas boa parte das colisões aqui vem dos nomes repetidos, e essas nenhum M resolve. Para mostrar os canis com M = 17 ou 31, use a opção 9.

4. Qual função espalhou melhor? A **ponderada** (função 2): com os mesmos 50 bichinhos e o mesmo M, o canil mais cheio caiu de 13 para 9 (M = 13) e de 11 para 7 (M = 17); a M = 13 nenhum canil ficou vazio; e houve menos colisões entre nomes diferentes (13 contra 14; 11 contra 12). Ela separa anagramas (Amor 784, Roma 731) e somas iguais (Frida 1050, Chico 1093), porque a posição de cada letra pesa. Conclusão tirada comparando as estatísticas lado a lado na opção 5.

6. Por que ordenar por microchip e não por nome? O microchip é único e numérico: cada bichinho tem uma posição certa na fila da saída, e comparar dois números é uma operação só. Os nomes se repetem (6 Lunas: qual vai para casa primeiro?) e comparar textos exige letra por letra, maiúsculas e acentos. E cada estrutura cuida de uma coisa: os canis (hash) acham pelo nome; a fila ordenada define a saída.

7. Quem venceu cada situação? Combina com $O(n^2)$ e $O(n \log n)$? Com 10.000: **aleatório**, Quick (0,55 ms e 158.262 comparações) contra Insertion (18 ms, 25 milhões) e Bubble (61 ms, 50 milhões); **já ordenado**, Bubble e Insertion (9.999 comparações, ~0,01 ms) e Quick em último (50 milhões, 37 ms); **inverso**, Quick foi o mais rápido em tempo (29 ms, contra 39 do Insertion e 57 do Bubble), mas os três fizeram 49.995.000 comparações, e o Quick muito menos trocas (5.000). Combina: de 1.000 para 10.000 (10×), as comparações do Bubble aleatório foram de 498.225 para 49.994.097 (100×, $O(n^2)$) e as do Quick de 10.111 para 158.262 (≈ 16×, perto dos ≈ 13× de $n \cdot \log_2 n$). As contagens mostram isso melhor que o tempo, que sofre com cache e JIT.

8. O Insertion mudou com dados já ordenados? Sim: **9.999 comparações e 0 deslocamentos** (n − 1), 0,016 ms contra 18 ms no aleatório. Cada bichinho compara uma vez com o vizinho da frente, vê que ele tem microchip menor e para: vira O(n), o melhor caso.

9. Algo estranho com o Quick? Sim: nos dados **já ordenados** e **inversos** ele fez 49.995.000 comparações (= n(n−1)/2) e, na fila já ordenada, levou 37 ms contra 0,006 ms do Bubble. O pivô é o último, que numa fila ordenada é sempre o maior microchip: a partição fica com n − 1 de um lado e nada do outro, então o "dividir ao meio" vira "diminuir 1". Além de $O(n^2)$, a recursão chega a 10.000 níveis e estourou a pilha (`StackOverflowError`) num teste com 512 KB; resolvemos rodando a corrida numa thread com pilha grande. Pivô do meio, sorteado ou mediana de três evitaria o problema.

10 Checklist de entrega

Código

✓ Tabela hash própria (canis), encadeamento, 2 funções
✓ Matricular, procurar (todos), ir para casa, mostrar canis
✓ Lotação e estatísticas; M = 13, 17, 31 (opção 9 troca M e função)
✓ 3 sorts, contagens, fila passo a passo ≤ 10
✓ Gerador 1.000/10.000; corrida 27 medições (n = 50 com os do arquivo)
✓ 48 testes JUnit verdes

Ainda com o grupo

☐ Trocar alguns dos 30 bichinhos pelos bichos reais de vocês
☐ Rodar a opção 8 no computador de vocês e usar esses tempos
☐ Ajustar mensagens e narração ao gosto do grupo (os testes da creche avisam se algo sumir)
☐ Cada um mexer em uma parte e rodar os testes

Relatório (3 a 6 páginas)

☐ Cenário B e decisões (encadeamento, inserir no início, as 2 definições de colisão, função ponderada, M = 31, lotação, TDD)
☐ Tabelas de tempo, comparações e trocas (opção 8)
☐ As 9 respostas acima, com os números de vocês